Dokumentacja Projektu: BuyBuddy

1. Opis aplikacji

BuyBuddy to nowoczesna aplikacja webowa służąca do zarządzania listami zakupów w czasie rzeczywistym. System pozwala użytkownikom na:

- **Zarządzanie kontem:** Rejestrację i logowanie przy użyciu nazwy użytkownika, adresu e-mail oraz hasła.
- **Tworzenie list:** Użytkownik będący właścicielem może tworzyć wiele niezależnych list zakupowych.
- **Współpracę:** Możliwość dodawania innych zarejestrowanych użytkowników do konkretnej listy, co pozwala na wspólne dodawanie i odznaczanie produktów.
- **Synchronizację Real-time:** Dzięki wykorzystaniu technologii WebSockets, zmiany wprowadzane przez jednego użytkownika są natychmiast widoczne dla pozostałych członków listy.

2. Podział prac w zespole

Prace nad projektem zostały podzielone równomiernie, uwzględniając kompetencje techniczne oraz organizacyjne:

- **Piotr Krzeszowski:**
  - Przygotowanie pełnej dokumentacji technicznej i projektowej.
  - Opracowanie scenariuszy testowych i wykonanie testów integracyjnych.
  - Konfiguracja środowiska konteneryzacji za pomocą narzędzia Docker.
  - Zrobienie dokumentacji
- **Krzysztof Springer:**
  - Implementacja interfejsu użytkownika w technologii React.
  - Projektowanie warstwy wizualnej i stylowanie z wykorzystaniem biblioteki SASS.
  - Zapewnienie responsywności aplikacji (RWD).
- **Maria Koczorowska:**
  - Projekt i implementacja serwera API w Node.js/Express.
  - Zarządzanie strukturą bazy danych MongoDB przy użyciu biblioteki Mongoose.
  - Implementacja logiki komunikacji real-time (Socket.io).

3. Opis bazy danych (Modele)

Aplikacja korzysta z nierelacyjnej bazy danych **MongoDB**. Struktura danych opiera się na trzech głównych schematach:

a) Model User (Użytkownik)

Przechowuje dane niezbędne do autoryzacji:

- `name`: Nazwa użytkownika (wymagana).
- `email`: Adres e-mail (unikalny).
- `password`: Hasło (zaszyfrowane).

b) Model List (Lista)

Zarządza strukturą listy i dostępami:

- `name`: Nazwa listy.
- `owner`: Referencja do modelu User (właściciel listy).
- `members`: Tablica referencji do modelu User (osoby zaproszone do listy).

c) Model Item (Produkt)

Definiuje elementy znajdujące się na liście:

- `name`: Nazwa produktu.
- `listId`: Referencja łącząca produkt z konkretną listą.

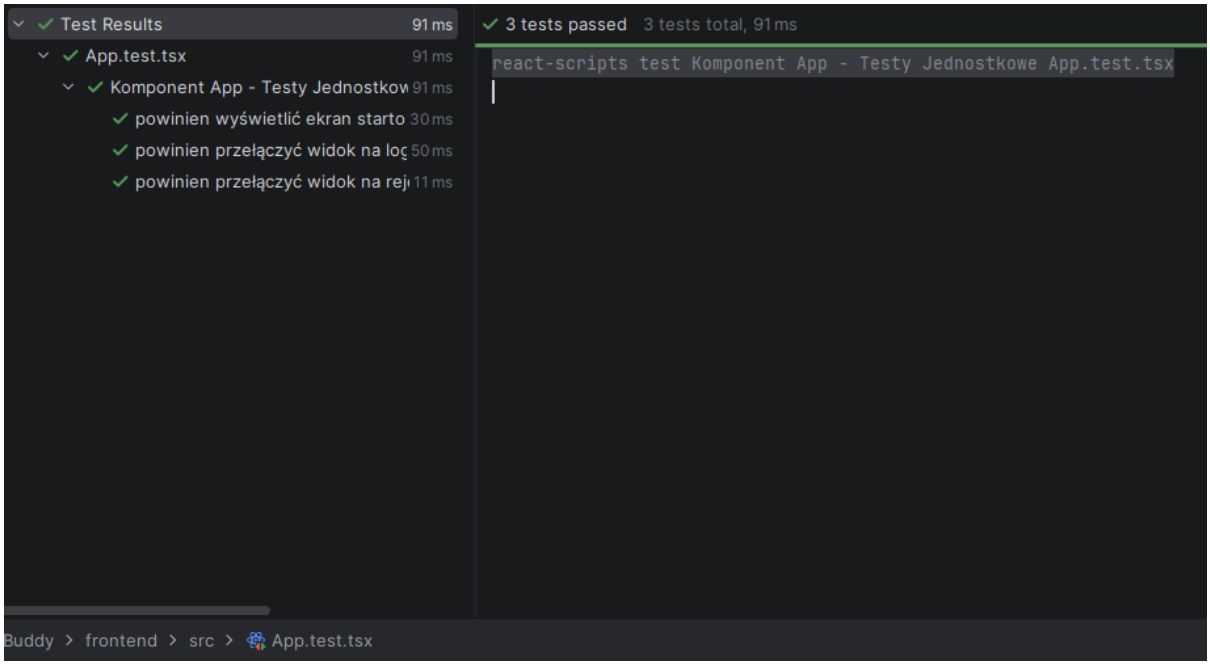
4. Specyfikacja techniczna

Projekt został zrealizowany w architekturze klient-serwer z wykorzystaniem następujących technologii:

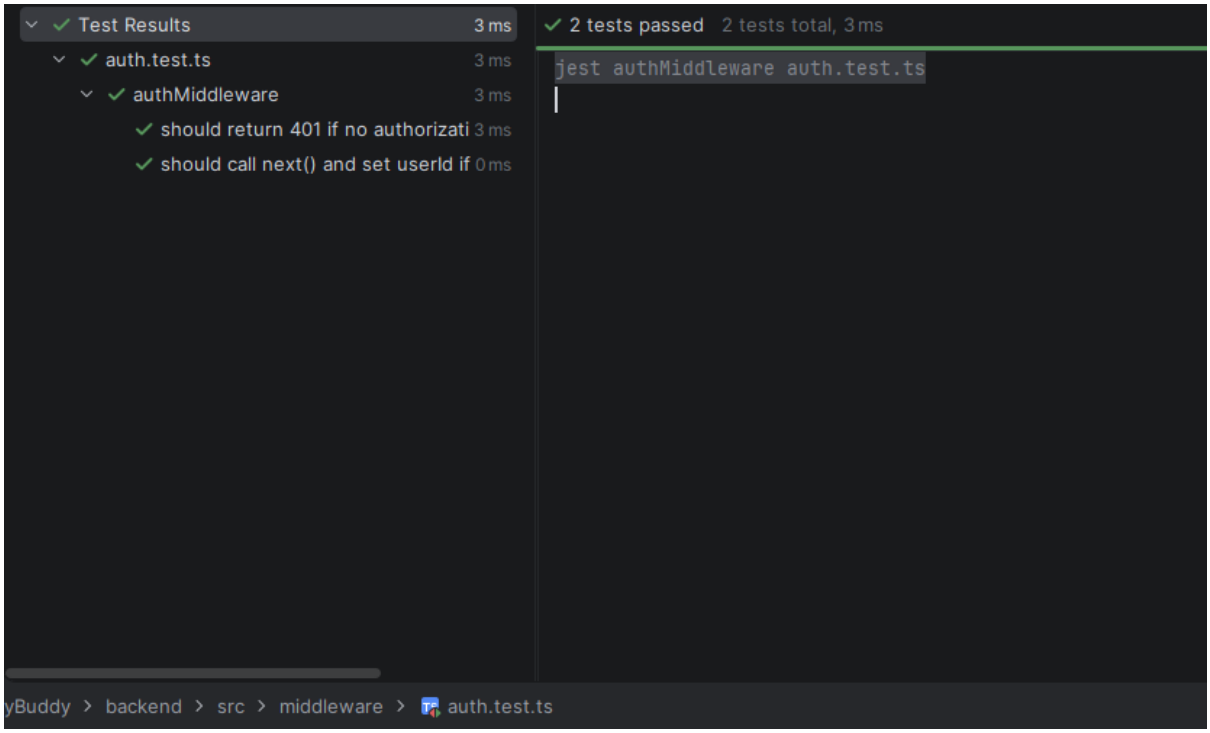
- **Backend:** Node.js (Express), MongoDB (Mongoose), Socket.io, TypeScript, JWT (JSON Web Token), Bcrypt.
- **Frontend:** React 19, Axios, Socket.io-client, SASS, TypeScript.
- **Infrastruktura:** Docker, Docker-Compose.

5. Scenariusze testowe i testy jednostkowe

- **Testy jednostkowe Frontend:** Przetestowano renderowanie głównego komponentu App oraz poprawność przełączania widoków logowania i rejestracji.



- **Testy jednostkowe Backend:** Przeprowadzono testy logiki autoryzacji (Middleware), weryfikując poprawne przypisywanie tokenów oraz blokowanie nieautoryzowanych żądań.



6. Weryfikacja środowiska kontenerowego (Docker)

W celu zapewnienia pełnej izolacji usług oraz powtarzalności środowiska uruchomieniowego, aplikacja **BuyBuddy** została w pełni skonteneryzowana przy użyciu narzędzia **Docker**. Proces orkiestracji usług odbywa się za pomocą pliku `docker-compose.yml`, który definiuje parametry pracy dla trzech kluczowych komponentów systemu.

Poniższa lista (potwierdzona komendą `docker ps`) przedstawia aktualnie uruchomione i poprawnie skonfigurowane kontenery:

- **buybuddy-frontend-1**: Serwer frontendu (React) pracujący na porcie **3001**.
- **buybuddy-backend-1**: Serwer API (Node.js) pracujący na porcie **3000**.
- **buybuddy-mongo-1**: Instancja bazy danych NoSQL (MongoDB) pracująca na standardowym porcie **27017**.

```
PS C:\Users\piotr\BuyBuddy\frontend> cd..

PS C:\Users\piotr\BuyBuddy> docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED         STATUS          PORTS                                          NAMES
9871228b9d6b   buybuddy-frontend  "docker-entrypoint.s…"   48 minutes ago  Up 48 minutes   0.0.0.0:3001->3000/tcp, [::]:3001->3000/tcp    buybuddy-frontend-1
3d87a41492f3   buybuddy-backend   "docker-entrypoint.s…"   48 minutes ago  Up 48 minutes   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp    buybuddy-backend-1
4c6719c487cf   mongo              "docker-entrypoint.s…"   48 minutes ago  Up 48 minutes   0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp buybuddy-mongo-1
```